

OpenEx

Digital Asset Exchange Simulation

DAY 15 PROJECT STATUS

OpenEx is a full-stack digital asset exchange simulation built to demonstrate how a modern cryptocurrency trading platform can be designed and developed. The project combines a Kotlin/Spring Boot backend, PostgreSQL database, React/Vite frontend, Python market-data service, WebSockets, and JWT authentication into one integrated trading platform.

The application simulates the core functionality of a digital asset exchange, including account authentication, wallets, order placement, order matching, trade execution, order books, market data, and a real-time trading dashboard.

Project Status: Full-Stack Integration

OpenEx has reached a working full-stack development stage. The main components of the application are integrated and can be run locally. The project is currently focused on demonstrating a complete exchange workflow from the user interface through the backend and database.

Core Components Integrated

- ✓ Spring Boot backend & PostgreSQL database
- ✓ React trading frontend
- ✓ Python market-data service
- ✓ JWT authentication
- ✓ Order management and matching
- ✓ Simulated BTC-USD market data
- ✓ WebSocket/STOMP infrastructure

Features & Functionality

1. Authentication & Security

OpenEx uses standard web security protocols to secure API access and manage user sessions.

- User login with JWT-based authentication.
- Password hashing with BCrypt.
- Protected backend endpoints demanding authorization headers.
- Authenticated order placement ensuring user validation.

2. Trading & Order Matching

The core matching engine processes market dynamics based on user orders.

- Support for Buy/Sell and Limit/Market orders.
- Order matching logic with support for partial order fills.
- Order cancellation and remaining quantity tracking.
- **Idempotency Keys** for order submission to prevent double-spending or duplicate trades.

3. Wallets & Account Management

Account wallets store simulated asset balances, handle deposits, and maintain persistent ledger updates after successful trades.

4. Order Book & Trade History

The application maintains an exchange-style order book containing active Buy/Sell orders, prices, and quantities. Executed orders generate trades that are populated as recent market executions, displaying the execution price, quantity, and trade time.

5. Market Data & WebSockets

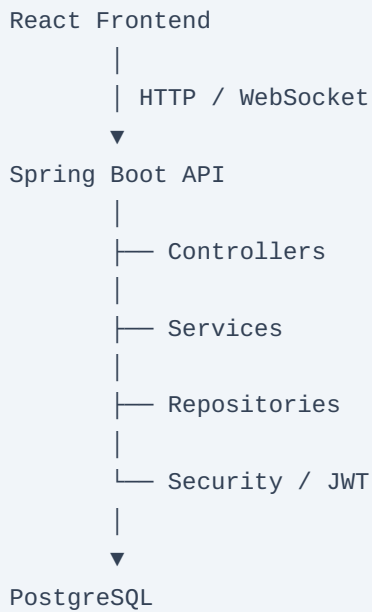
A Python market-data service provides simulated cryptocurrency market data (BTC-USD). WebSocket/STOMP infrastructure guarantees live updates for market prices, order books, trade events, and price changes.

Technology Stack

Domain	Technologies
Backend	Kotlin, Spring Boot, Spring Security, Spring Data JPA, Hibernate, JWT, BCrypt, WebSockets/STOMP, Gradle
Database	PostgreSQL, Flyway Migrations, Docker
Frontend	React, Vite, JavaScript, CSS, Fetch API, Recharts
Market Data Service	Python, Flask, Simulated market data, Technical indicators

System Architecture

The backend follows a layered architecture, communicating dynamically with the frontend through REST APIs and WebSockets.



Application Structure (Directory Tree)

```
Openx3.0/
├── frontend/
│   ├── src/
│   │   ├── components/ (Login, OrderForm, MyOrders, MarketChart)
│   │   ├── hooks/ (useMarketData.js)
│   │   └── App.jsx
│   └── package.json / vite.config.js
├── python-service/
│   ├── app.py
│   └── market_simulator.py
├── src/
│   └── main/kotlin/com/openex/core/
│       ├── api/ | config/ | domain/ | repository/ | service/
└── build.gradle.kts / settings.gradle.kts
```

Application Workflows

Trading Flow

1. User signs in and Backend validates credentials.
2. JWT token is returned and Frontend stores it.
3. User navigates to BTC-USD market and selects BUY or SELL.

4. User enters price and quantity, then frontend sends order to backend.
5. Backend validates the order and it enters the matching engine.
6. Matching orders are found and trade is executed.
7. Wallets and order balances are updated.
8. Trade appears in the user's trade history.

Authentication Flow

```
User -> [email + password] -> Login API
Login API -> [validate credentials] -> JWT Token
JWT Token -> Frontend
Frontend -> [Authorization: Bearer ] -> Protected API
```

API Reference Examples

Standard RESTful endpoints are utilized. The backend handles payload processing and state mutation securely.

Login

POST /api/auth/login

```
{
  "email": "alice@openex.test",
  "password": "password"
}
```

Create Order

POST /orders

```
{
  "symbol": "BTC-USD",
  "side": "SELL",
  "type": "LIMIT",
  "price": 3000,
  "quantity": 0.1
}
```

Note: Authenticated requests require the Authorization: Bearer <JWT_TOKEN> header. An idempotency key is also highly recommended.

Running the Project

Follow these steps to spin up the local development environment:

1. **Start PostgreSQL:** Ensure Docker Desktop is running and start the database container (verify with `docker ps`).
2. **Start the Backend:** Run `./gradlew bootRun` from the root directory. Backend runs on `http://localhost:8080`.
3. **Start the Frontend:** Open a new terminal, navigate to `cd frontend`, run `npm install`, then `npm run dev`. Frontend runs on `http://localhost:5173`.
4. **Start Market Service:** From `cd python-service`, run `python app.py` to stream simulated market data.

Testing: Backend tests can be run via `./gradlew test`. A frontend production build can be generated using `npm run build`.

Educational Disclaimer

OpenEx is an educational trading simulation. It does not execute real cryptocurrency transactions, manage real funds, or connect to a live financial exchange. All market prices, order books, and trading activities are strictly simulated for development and demonstration purposes.